SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA0301	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)		
Student Name:	S. Bramhani	**Reg. No.:**	192425319
Section / Batch:	B.Tech - AIML	**Date:**	6/11/2026

Code of Conduct

I S. Bramhani (192425319) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Bramhani

Instructions

- Read each question carefully before answering.
- Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
					TOTAL MARKS	

Question 1: Regular Expression-Based Text Processing BTL 3 – Apply | 10 Marks

Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

Regular Expression (RE) is a method used to identify and extract specific patterns from a given text. It is commonly used in Natural Language Processing (NLP) for text searching, validation and information extraction.

Python provides the `re` module to perform regular expression operations. Using this module, we can extract email addresses, phone numbers, capitalized words, numerical values and words ending with particular suffixes.

Regular expressions help process large amounts of text quickly without checking every word manually.

- Regular expressions are used for pattern matching in text.
- The `re` module provides functions for searching and extracting patterns.
- `re.findall()` extracts all matching patterns from the text.
- `\d` represents a digit, while `[A-Z]` represents an uppercase letter.
- `\b` identifies word boundaries.
- Regular expressions are useful in email validation, phone number extraction and text processing.

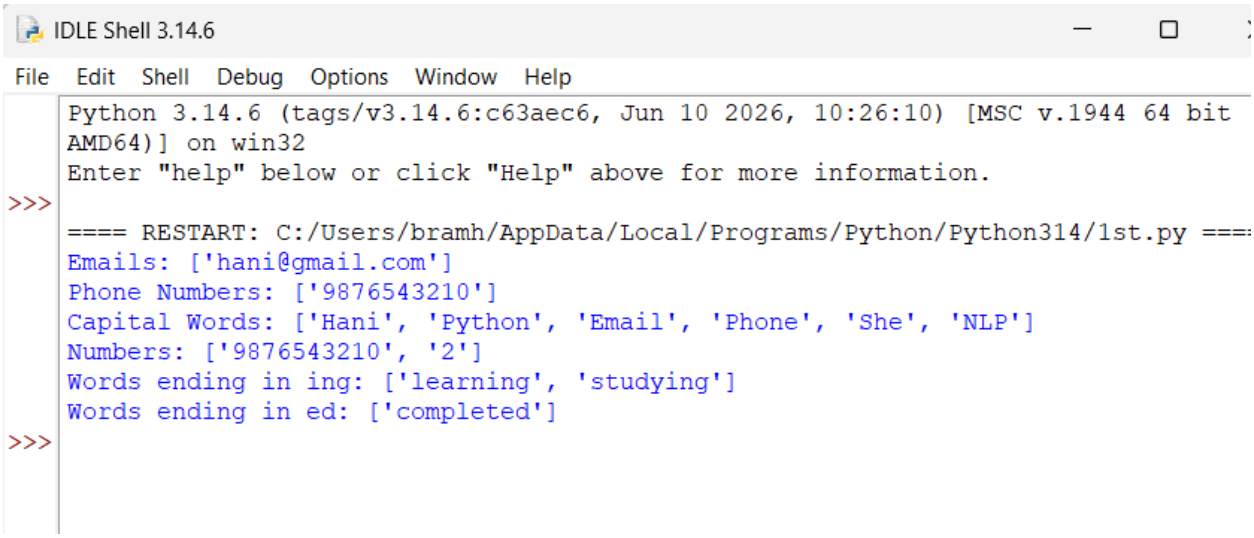
Python Program

```
import re

text = """Hani is learning Python.
Email: hani@gmail.com
Phone: 9876543210.
She completed 2 projects and is studying NLP."""

email = re.findall(r'[\w.-]+@[\w.-]+\.\w+', text)
phone = re.findall(r'\b\d{10}\b', text)
capital = re.findall(r'\b[A-Z][a-zA-Z]*\b', text)
numbers = re.findall(r'\b\d+\b', text)
ing = re.findall(r'\b\w+ing\b', text)
ed = re.findall(r'\b\w+ed\b', text)
```

```
print("Emails:", email)
print("Phone Numbers:", phone)
print("Capital Words:", capital)
print("Numbers:", numbers)
print("Words ending in ing:", ing)
print("Words ending in ed:", ed)
```



Example

Input: `Ravi is studying Python and completed 3 projects.`

Feature	Extracted Result
Capitalized words	Ravi, Python
Numerical values	3
Words ending in ing	studying
Words ending in ed	completed

Result: The program successfully extracts different language features from the given paragraph using regular expressions.

Question 2: Finite State Automata for Word Pattern Recognition BTL 3 – Apply | 10 Marks

Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

Finite State Automata (FSA) is a computational model used to recognize whether a given string follows a particular pattern. It consists of a finite number of states and transitions between those states.

An automaton reads the input characters one by one and changes its state according to the given conditions. After processing the complete word, it checks whether the final state is an accepting state or a rejecting state.

In this program, an English word is accepted only when it satisfies all three conditions:

- The word must start with a vowel.
- The word must end with a consonant.
- The word must contain at least two consecutive vowels.

- ❖ FSA is used for recognizing patterns in strings.
- ❖ A state represents the current condition of the input.
- ❖ A transition represents movement from one state to another.
- ❖ The initial state is where the automaton starts.
- ❖ The final state determines whether the word is accepted.
- ❖ Vowels are `a, e, i, o, u`; the remaining English alphabet letters are consonants.

Python Program

```
vowels = "aeiou"

def check(word):
  word = word.lower()

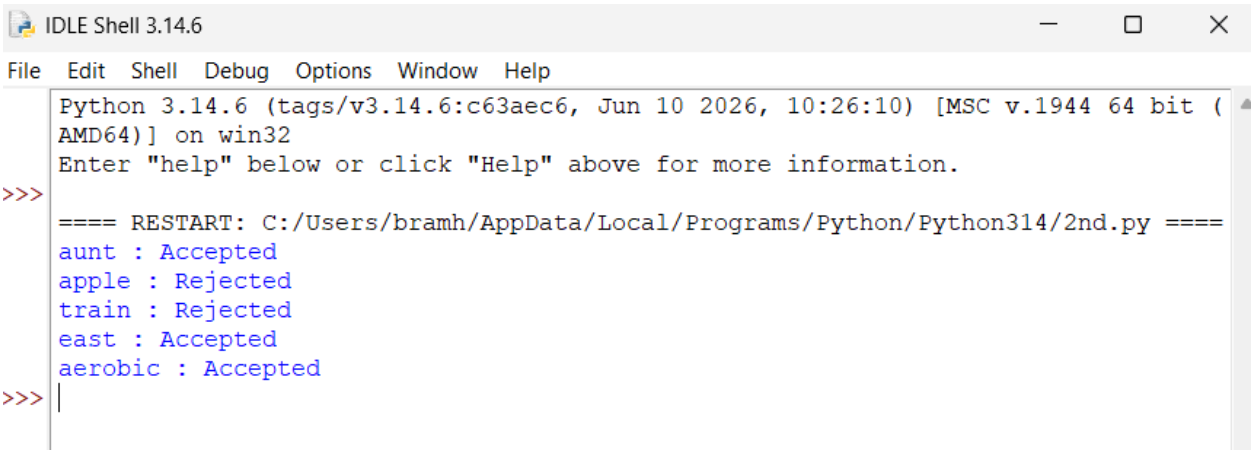
  if not word.isalpha():
    return "Rejected"

  start = word[0] in vowels
  end = word[-1] not in vowels
  double = any(word[i] in vowels and word[i+1] in vowels
          for i in range(len(word)-1))

  if start and end and double:
    return "Accepted"
  return "Rejected"

words = ["aunt", "apple", "train", "east", "aerobic"]

for word in words:
  print(word, ":", check(word))
```



Example

Word	Starting Character	Consecutive Vowels	Ending Character	Result
Aunt	Vowel	Yes	Consonant	Accepted
Apple	Vowel	Yes	Vowel	Rejected
Train	Consonant	No	Consonant	Rejected
East	Vowel	Yes	Consonant	Accepted

Result: The program recognizes English word patterns by checking the starting character, consecutive vowels and ending character.

Question 3: Morphological Analysis of English Words BTL 4 – Analyze | 10 Marks

Develop a Python program to perform morphological analysis on a given set of English

words.

1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

Morphological Analysis is the process of studying the internal structure of words by dividing them into smaller meaningful units called morphemes. A morpheme is the smallest unit of a language that carries meaning.

English words are formed using root words and affixes. Affixes are additional parts added to a root word to modify its meaning or grammatical function. They are classified into prefixes and suffixes.

A prefix is added at the beginning of a root word, whereas a suffix is added at the end. Morphological analysis helps in understanding how words are formed and how their meanings change.

For example, the word unhappiness consists of the prefix `un`, the root `happy` and the suffix `ness`.

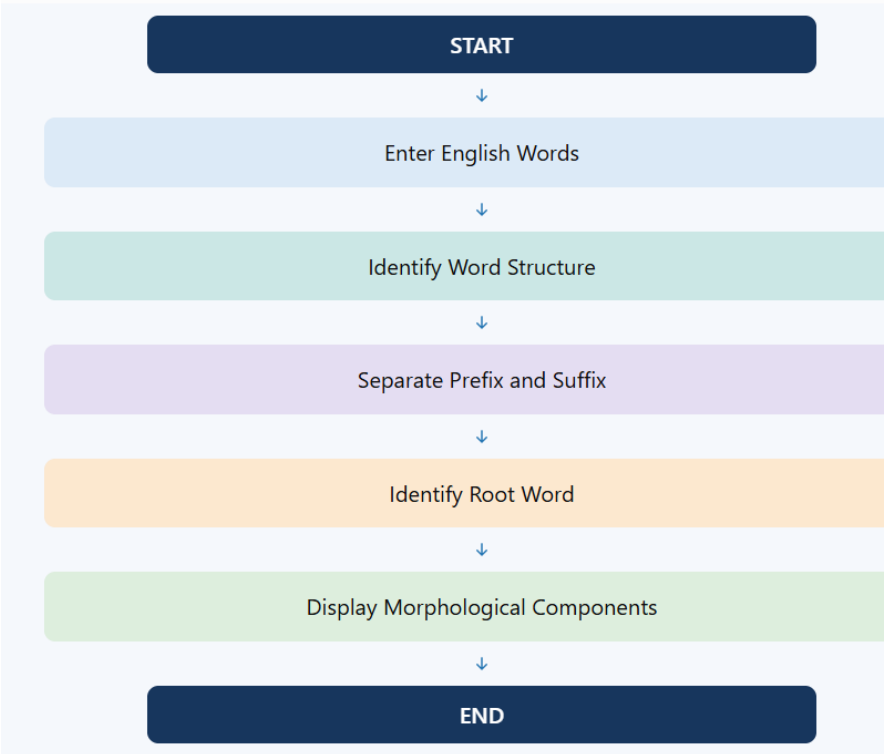
- Morphology is the study of word formation and structure.
- A root word carries the main meaning of a word.
- A prefix is added before the root word.
- A suffix is added after the root word.
- Affixes can change the meaning or grammatical category of a word.
- Morphological analysis is useful in NLP applications such as text processing, search engines and language translation.

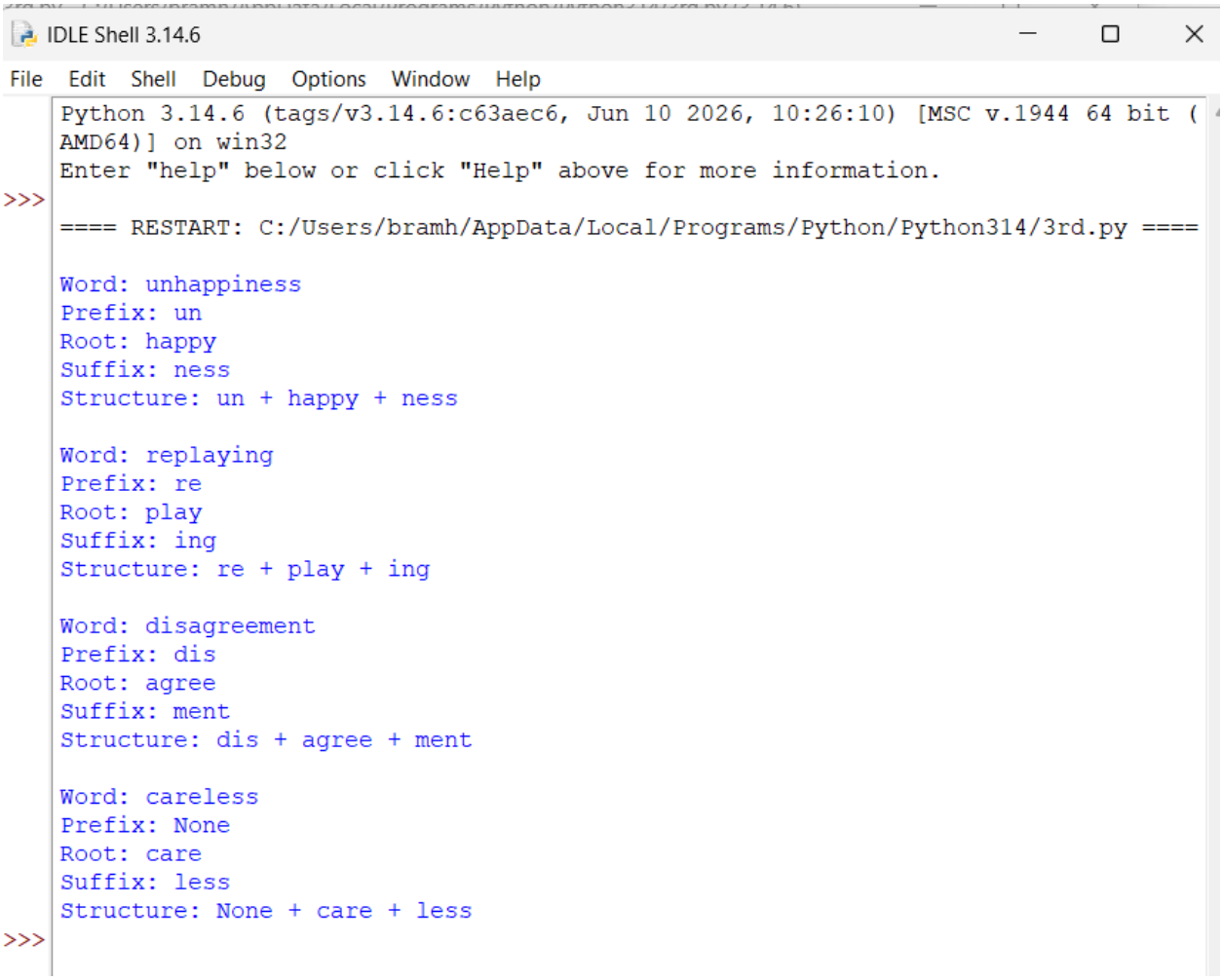
Python Program

```
words = {
  "unhappiness": ("un", "happy", "ness"),
  "replaying": ("re", "play", "ing"),
  "disagreement": ("dis", "agree", "ment"),
  "careless": ("None", "care", "less")
}

for word, parts in words.items():
  prefix, root, suffix = parts

  print("\nWord:", word)
  print("Prefix:", prefix)
  print("Root:", root)
  print("Suffix:", suffix)
  print("Structure:", prefix, "+", root, "+", suffix)
```





Example

Word	Prefix	Root	Suffix
Unhappiness	Un	Happy	Ness
Replaying	Re	Play	Ing
Disagreement	Dis	Agree	Ment
Careless	None	Care	Less

Result: The program separates English words into prefixes, root words and suffixes to understand their morphological structure.

Question 4: Inflectional and Derivational Morphology BTL 3 – Apply | 10 Marks

Write a Python program to identify and classify inflectional and derivational word forms

from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.
4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

Inflectional and Derivational Morphology are two important processes used in English word formation.

Inflectional morphology modifies the grammatical form of an existing word without changing its basic meaning or grammatical category. It is mainly used to represent tense, number and comparison.

For example, `book` becomes `books` to indicate plural, and `play` becomes `played` to indicate past tense.

Derivational morphology creates a new word by adding prefixes or suffixes to an existing root word. It may change the meaning or grammatical category of the original word.

For example, `happy` becomes `unhappy` by adding a prefix, and `care` becomes `careless` by adding a suffix.

- Inflectional morphology changes the grammatical form of a word.
- Derivational morphology creates new words.
- Inflectional suffixes include `-s`, `-es`, `-ed` and `-ing`.
- Derivational prefixes include `un-`, `re-` and `dis-`.
- Derivational suffixes include `-ness`, `-ful`, `-less` and `-er`.
- Inflectional morphology generally maintains the original word category, whereas derivational morphology may change it.

Python Program

```
import re

text = "Cats are playing. Boys walked and teachers are helpful."

words = re.findall(r'\b[a-zA-Z]+\b', text.lower())



for word in words:

    if word.endswith("ies") or word.endswith("s"):

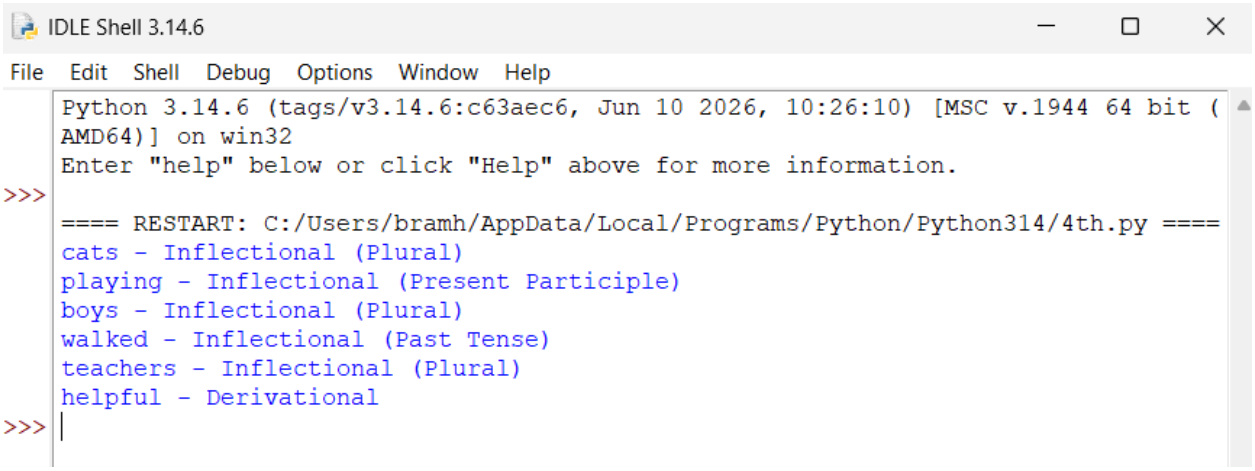
        print(word, "- Inflectional (Plural)")

    elif word.endswith("ed"):

        print(word, "- Inflectional (Past Tense)")

    elif word.endswith("ing"):
```

```
        print(word, "- Inflectional (Present Participle)")
    elif word.endswith(("ful", "less", "ness")) or word.startswith(("un", "re", "dis")):
        print(word, "- Derivational")
```



Example

Original Word	Modified Word	Classification
Book	Books	Inflectional
Play	Played	Inflectional
Play	Playing	Inflectional
Happy	Unhappy	Derivational
Care	Careless	Derivational
Teach	Teacher	Derivational

Result: The program identifies and classifies word forms into inflectional and derivational categories based on their prefixes and suffixes.

Question 5: Porter Stemming Algorithm BTL 4 – Analyze | 10 Marks

Implement Porter Stemming Algorithm using Python to identify root forms of English words.

1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.

Analyze how stemming reduces different word forms

Example words: playing, played, studies, studying, connected

Porter Stemming is a word-processing technique used in Natural Language Processing to reduce different forms of a word to a common stem by removing certain suffixes.

The Porter Stemming Algorithm was developed by Martin Porter in 1980. It applies a series of rules to remove common English word endings.

For example, words such as `playing` and `played` are reduced to `play`. Similarly, `connected` and `connection` are reduced to `connect`.

Stemming is useful in information retrieval and search engines because it helps identify different forms of the same word during searching.

- Stemming reduces words to their common stem.
- The Porter algorithm removes common suffixes using predefined rules.
- It helps reduce the number of different word forms.
- It is used in search engines and document retrieval.
- Different words may produce the same stem.
- A stemmed word is not always a complete dictionary word.

Installation

```
pip install nltk
```

Python Program

```
from nltk.stem import PorterStemmer
from collections import defaultdict

ps = PorterStemmer()

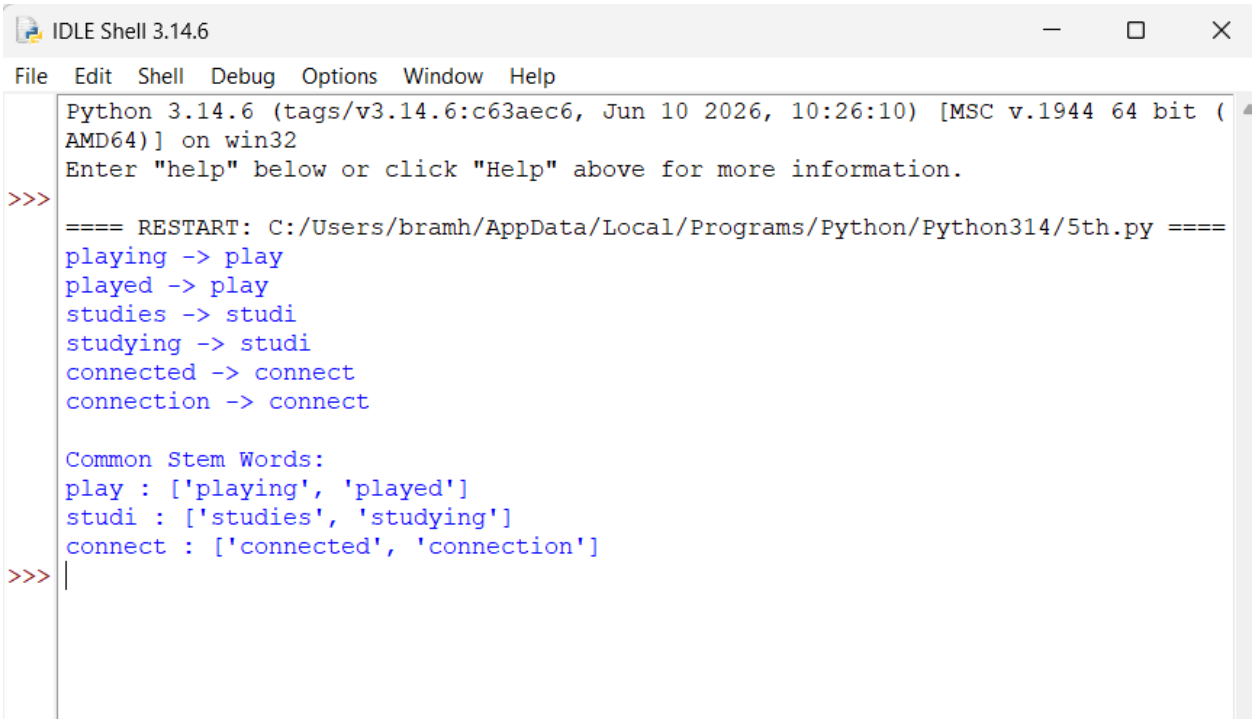
words = ["playing", "played", "studies",
      "studying", "connected", "connection"]

groups = defaultdict(list)

for word in words:
  stem = ps.stem(word)
  print(word, "->", stem)
  groups[stem].append(word)

print("\nCommon Stem Words:")

for stem, original in groups.items():
  if len(original) > 1:
    print(stem, ":", original)
```



Example

Original Word	Stemmed Word
Playing	Play
Played	Play
Studies	Studi
Studying	Studi
Connected	Connect
Connection	Connect

Result: The program applies the Porter Stemming Algorithm to reduce different English word forms to their common stems. This makes text searching and document comparison easier.

Question 6: Implementation of N-gram Language Models BTL 3 – Apply | 10 Marks

Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

An N-gram Language Model is a statistical model used in Natural Language Processing to study the sequence of words in a sentence and predict the next word based on previous words.

An N-gram is a group of consecutive words taken from a given text. Depending on the number of words considered, N-grams are classified into unigram, bigram and trigram models.

A unigram model considers one word at a time, a bigram model considers two consecutive words, and a trigram model considers three consecutive words.

These models are used in applications such as text prediction, speech recognition, spelling correction and machine translation.

- Unigram represents a sequence of one word.
- Bigram represents a sequence of two consecutive words.
- Trigram represents a sequence of three consecutive words.
- N-gram frequency indicates how many times a particular sequence occurs.
- Probability is used to estimate the likelihood of a word appearing after another word.
- Higher-order N-grams provide more information about the surrounding context.

Python Program

```
from collections import Counter

text = "I love Python and I love learning Python"

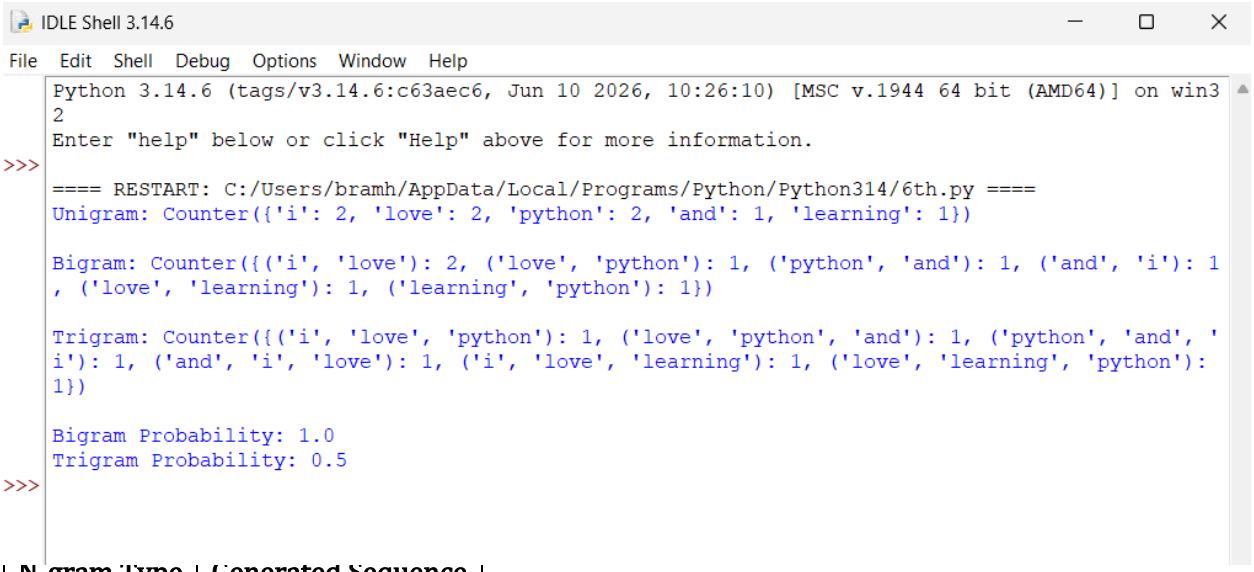
words = text.lower().split()

unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
trigram = Counter(zip(words, words[1:], words[2:]))

print("Unigram:", unigram)
print("\nBigram:", bigram)
print("\nTrigram:", trigram)

p1 = bigram[("i", "love")] / unigram["i"]
p2 = trigram[("i", "love", "learning")] / bigram[("i", "love")]

print("\nBigram Probability:", p1)
print("Trigram Probability:", p2)
```



N-gram Type	Generated Sequence
Unigram	I, love, NLP
Bigram	I love, love NLP
Trigram	I love NLP

Result: The program generates unigram, bigram and trigram sequences, calculates their frequencies and determines the probabilities of selected word sequences.

Question 7: Word Frequency and N-gram Smoothing BTL 3 – Apply | 10 Marks

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

Word Frequency refers to the number of times a particular word appears in a given text corpus. It helps identify commonly used words and understand the distribution of words in a document.

In an N-gram language model, bigram frequency is used to calculate the probability of one word appearing after another word.

However, some word combinations may not appear in the training text. Their probability becomes zero, which creates a problem during language prediction.

To solve this problem, smoothing techniques are used. Laplace Smoothing, also known as Add-One Smoothing, adds one to every bigram count so that unseen word combinations receive a non-zero probability.

- Word frequency counts the occurrences of individual words.
- Bigram frequency counts consecutive pairs of words.
- Unsmoothed probability is calculated using the original frequency counts.
- An unseen bigram receives zero probability without smoothing.
- Laplace smoothing adds one to each bigram count.
- Vocabulary size is represented by V in the smoothing formula.

Unsmoothed Bigram Probability

$$P(w_2|w_1) = \frac{C(w_1, w_2)}{C(w_1)}$$

Laplace Smoothing

$$P(w_2|w_1) = \frac{C(w_1, w_2) + 1}{C(w_1) + V}$$

Where:

- $C(w_1, w_2)$ = Bigram frequency.
- $C(w_1)$ = Frequency of the first word.
- V = Number of unique words in the vocabulary.

Python Program

```
from collections import Counter

text = "I love NLP and I love Python"

words = text.lower().split()

unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
```

```
V = len(unigram)

w1, w2 = "love", "python"

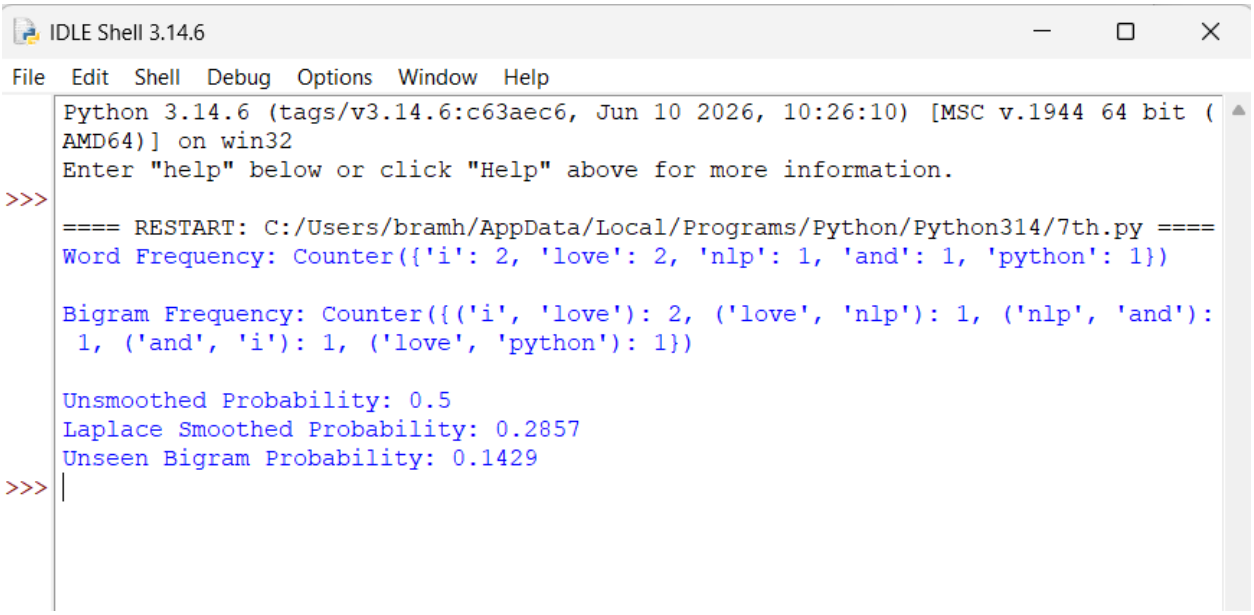
count = bigram[(w1, w2)]

p1 = count / unigram[w1]
p2 = (count + 1) / (unigram[w1] + V)

print("Word Frequency:", unigram)
print("\nBigram Frequency:", bigram)

print("\nUnsmoothed Probability:", p1)
print("Laplace Smoothed Probability:", round(p2, 4))

# Unseen bigram
p3 = 1 / (unigram["love"] + V)
print("Unseen Bigram Probability:", round(p3, 4))
```



Example:

Consider the bigram `love Python`.

Given:

- Bigram frequency = 1
- Frequency of `love` = 2
- Vocabulary size = 5

Without smoothing:

$$P = \frac{1}{2} = 0.5$$

With Laplace smoothing:

$$P = \frac{1+1}{2+5} = \frac{2}{7} = 0.2857$$

Result: Laplace smoothing assigns non-zero probabilities to unseen bigrams and helps improve the reliability of language models.

Question 8: Entropy Calculation for Language Models BTL 4 – Analyze | 10 Marks

Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.
5. Analyze how the context of preceding words influences the uncertainty of the next word.

Entropy is a measure of uncertainty or unpredictability in a language model. It indicates how difficult it is to predict the next word in a given text.

Shannon's Entropy Formula is used to calculate the uncertainty of a probability distribution. In NLP, entropy helps understand how predictable words are in a language.

A unigram model calculates probabilities using individual word frequencies, whereas a bigram model considers the previous word while predicting the next word.

By considering the previous word, a bigram model can reduce uncertainty because it uses additional context from the sentence.

- Entropy measures the uncertainty in a probability distribution.
- It is calculated using Shannon's entropy formula.
- Unigram entropy is based on individual word probabilities.
- Bigram conditional entropy considers the previous word.
- Lower entropy indicates greater predictability.
- Higher entropy indicates greater uncertainty.
- Entropy is commonly measured in bits when logarithm base 2 is used.

Shannon's Entropy

$$H(X) = -\sum_{x} P(x) \log_2 P(x)$$

Bigram Conditional Entropy

$$H(W_2|W_1) = -\sum_{w_1,w_2} P(w_1, w_2) \log_2 P(w_2|w_1)$$

Where:

- $P(x)$ = Probability of a word.
- $P(w_1, w_2)$ = Probability of a word pair.
- $P(w_2|w_1)$ = Probability of the next word given the previous word.

Python Program

```
import math
from collections import Counter

text = "I like NLP I like Python I love NLP I love Python"

words = text.lower().split()

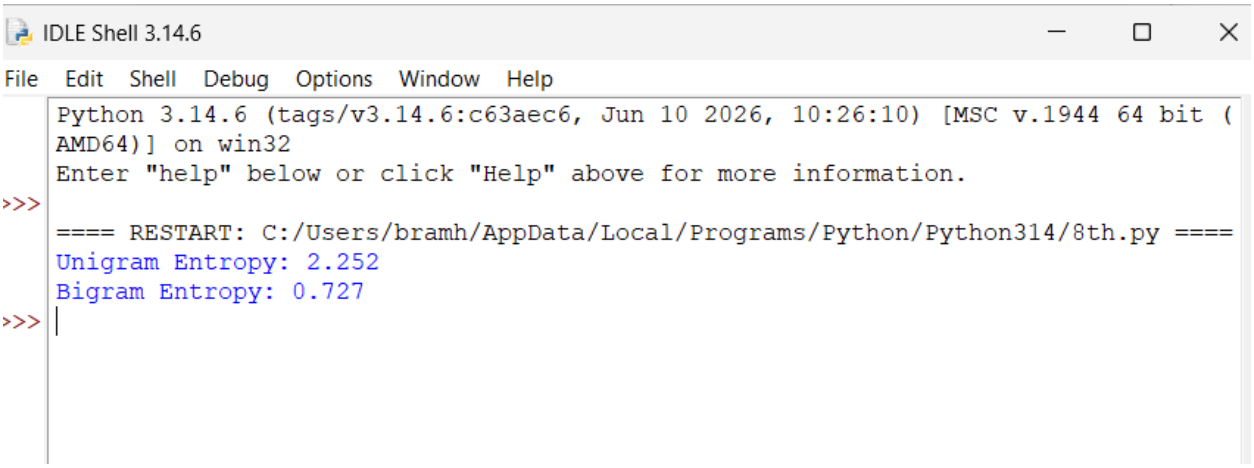
unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
```

```
# Unigram Entropy
H1 = 0
for count in unigram.values():
   p = count / len(words)
   H1 -= p * math.log2(p)

# Bigram Conditional Entropy
H2 = 0
total = len(words) - 1
context = Counter(words[:-1])

for (w1, w2), count in bigram.items():
   p = count / total
   conditional = count / context[w1]
   H2 -= p * math.log2(conditional)

print("Unigram Entropy:", round(H1, 3))
print("Bigram Entropy:", round(H2, 3))
```



Example

Consider the sentence:

```
I love NLP
```

- Unigram: I, love, NLP
- Bigram: I love, love NLP

The unigram model calculates the probability of each word independently. The bigram model considers the previous word when calculating the probability of the next word.

Question 9: English Part-of-Speech Tagging BTL 3 – Apply | 10 Marks

Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of

each POS category. Suggested Python library:
NLTK.

Part-of-Speech (POS) Tagging is the process of identifying the grammatical category of each word in a sentence. It assigns a specific grammatical tag to every word based on its role and usage in the sentence.

The main POS categories include nouns, verbs, adjectives, adverbs, prepositions, pronouns and conjunctions.

Python provides the Natural Language Toolkit (NLTK) library to perform POS tagging. It uses a trained English language model to identify the grammatical role of words.

POS tagging is an important step in NLP applications such as text analysis, machine translation, speech recognition and information extraction.

- POS tagging identifies the grammatical role of each word.
- NLTK is a Python library used for NLP operations.
- word_tokenize() divides a sentence into individual words.
- pos_tag() assigns grammatical tags to the words.
- Each word is assigned a suitable POS tag.
- POS tags help computers understand the grammatical structure of sentences.

POS Tag	Meaning	Example
NN	Singular Noun	Student
NNS	Plural Noun	Books
NNP	Proper Noun	Ravi
NNPS	Proper Plural Noun	Indians
VB	Base Verb	Read
VBD	Past Tense Verb	Played
VBG	Present Participle	Playing
JJ	Adjective	Intelligent
RB	Adverb	Quickly
IN	Preposition	In, On, At
PRP	Pronoun	She, They
DT	Determiner	The, A

Python Program

```
import nltk

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')

text = "The intelligent student reads books quickly."

words = nltk.word_tokenize(text)

tags = nltk.pos_tag(words)

count = {

    "Noun": 0, "Verb": 0, "Adjective": 0,

    "Adverb": 0, "Preposition": 0

}

print("Word\tPOS Tag")

for word, tag in tags:

    print(word, "\t", tag)

    if tag.startswith("NN"):

        count["Noun"] += 1

    elif tag.startswith("VB"):

        count["Verb"] += 1

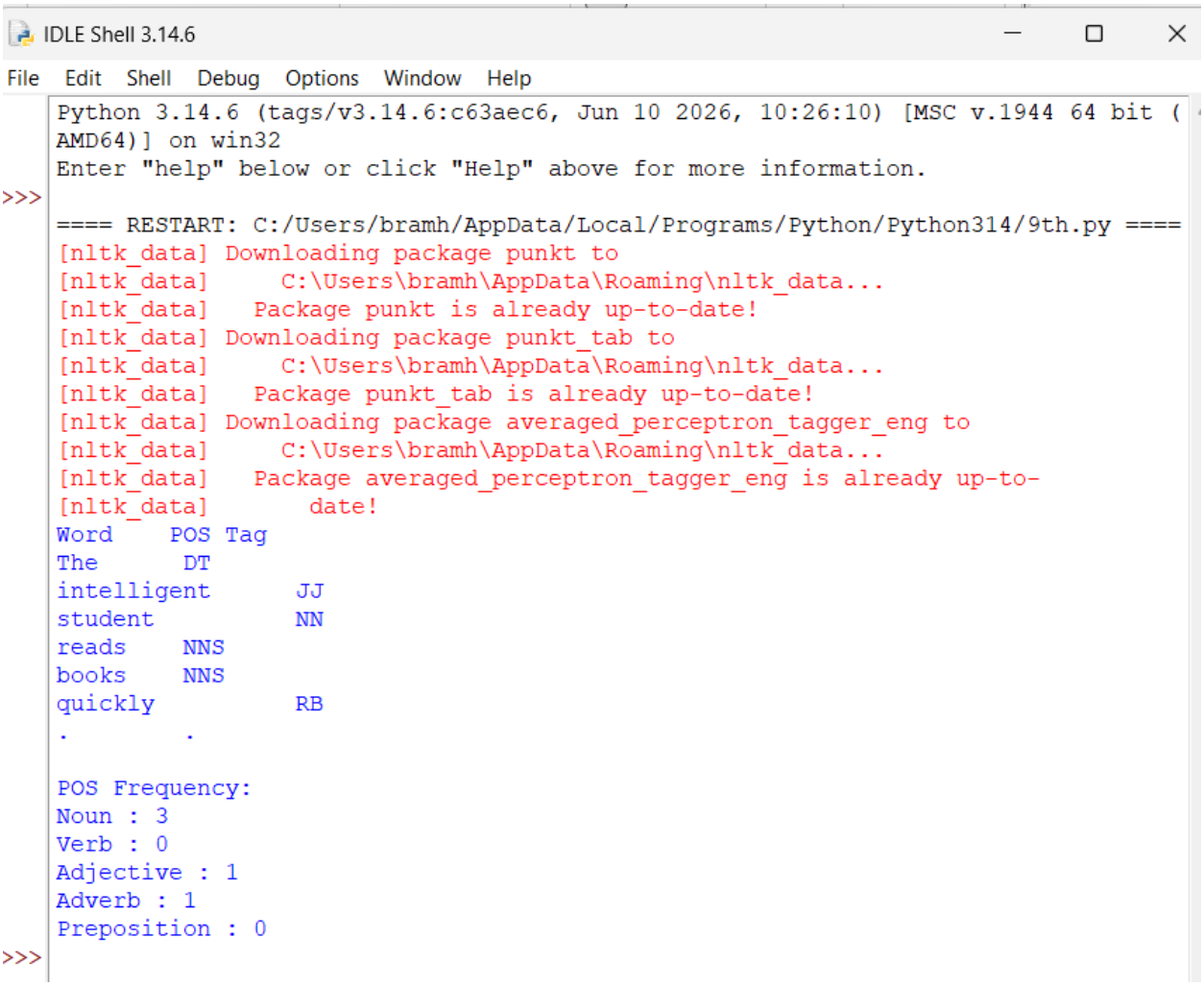
    elif tag.startswith("JJ"):

        count["Adjective"] += 1

    elif tag.startswith("RB"):
```

```
        count["Adverb"] += 1
    elif tag == "IN":
        count["Preposition"] += 1

print("\nPOS Frequency:")
for category, total in count.items():
        print(category, ":", total)
```



Question 10: Comparative Analysis of POS Tagging Techniques BTL 4 –
Analyze | 10 Marks

Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.

Suggested Python libraries: NLTK and spaCy.

POS Tagging techniques are used in Natural Language Processing to identify the grammatical category of words in a sentence. Different techniques use different methods to assign the correct grammatical tags.

The three important POS tagging techniques are Rule-Based Tagging, Stochastic Tagging and Transformation-Based Tagging.

Rule-Based Tagging: This technique uses predefined grammatical rules to identify the POS tag of a word. For example, words ending in `-ly` are generally identified as adverbs.

Stochastic Tagging: This technique uses statistical methods and probability to identify the most likely POS tag. It learns word patterns from previously tagged training data.

Transformation-Based Tagging: This technique initially assigns POS tags to words and then applies correction rules to improve the tagging results. It changes incorrect tags based on the surrounding words and grammatical context.

These techniques can be compared by applying them to the same sentences and observing the differences in their tagging results.

Python Program

```
import nltk

from nltk.corpus import brown

from nltk.tag import UnigramTagger, DefaultTagger



nltk.download('brown')

nltk.download('punkt')
```

```
nltk.download('punkt_tab')


# Training data
data = brown.tagged_sents()[:5000]


# Input sentence
sentence = "They want to play cricket quickly."
words = nltk.word_tokenize(sentence)


# 1. Rule-Based Tagger
rules = [
    (r'.*ly$', 'RB'),
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD')
]


rule = nltk.RegexpTagger(rules, backoff=DefaultTagger('NN'))


# 2. Stochastic Tagger
stochastic = UnigramTagger(
    data, backoff=DefaultTagger('NN')
)
```

```
# 3. Transformation-Based Tagger

initial = stochastic.tag(words)


transformed = initial[:]


for i in range(1, len(words)):

    if words[i-1].lower() == "to" and transformed[i][1] == "NN":

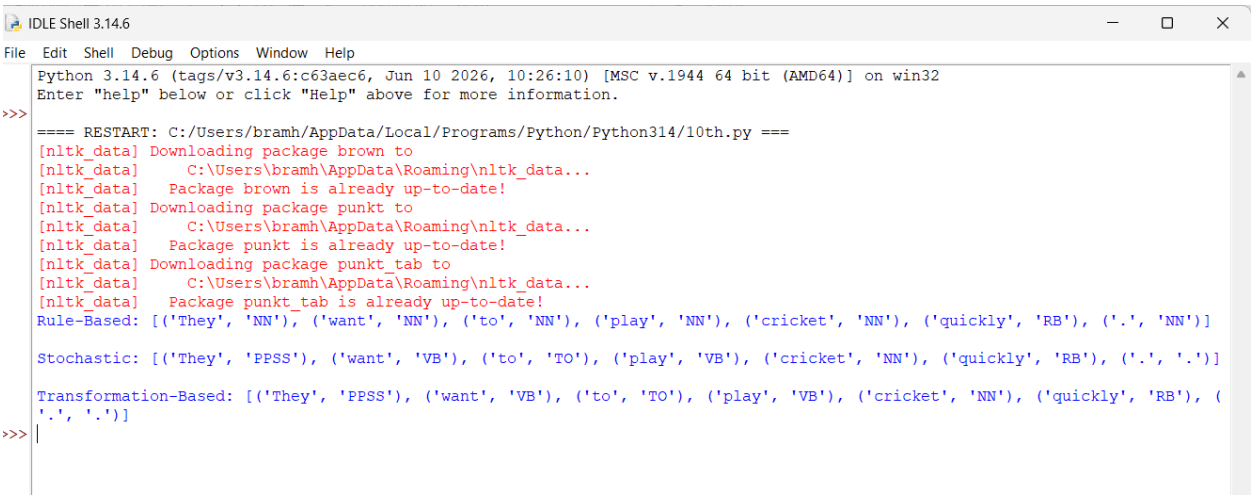
        transformed[i] = (words[i], "VB")


# Display Results

print("Rule-Based:", rule.tag(words))

print("\nStochastic:", initial)

print("\nTransformation-Based:", transformed)
```



0

Question	Major Concept	BTL Level	Marks
1	Regular Expressions	3	10
2	Finite State Automata	3	10
3	Morphological Analysis	4	10
4	Inflectional and Derivational Morphology	3	10
5	Porter Stemmer	4	10
6	N-gram Models	3	10
7	Word Counting and Smoothing	3	10
8	Entropy	4	10
9	POS Tagging	3	10
10	POS Tagging Techniques	4	10

Evaluation Rubric (10 Marks per Assignment)

Evaluation Criteria	Marks
Understanding of the NLP Concept	2
Python Implementation and Logic	3
Correctness of Output	2
Analysis and Interpretation of Results	2
Code Readability and Documentation	1
Total	10